

ISM3232 — Week 2 Lab

zsh Navigation & File Operations — Instructor Facilitation Guide

ISM3232 · Business Application Development · USF Muma College of Business

Module 02 · Unit 1 · Developer Foundations

Contents

1	Session Snapshot	2
2	Learning Objectives	2
3	Before Class — Setup Checklist	3
4	Materials Needed	3
5	Timing Plan (75 minutes)	3
6	Segment-by-Segment Walkthrough	4
6.1	Setup (0:00–0:04)	4
6.2	Part 1 — Navigation (0:04–0:20, 16 min)	4
6.3	Part 2 — File Operations (0:20–0:38, 18 min)	7
6.4	Part 3 — Safe File Deletion (0:38–0:53, 15 min)	9
6.5	Part 4 — Command Reference README (0:53–1:08, 15 min)	11
6.6	Stretch — Pipes and Redirect (1:08–1:15, as time allows)	13
7	Wrap-Up (last ~7 minutes)	14
8	Appendix A — Full Command Reference	14
9	Appendix B — Extra Practice (only if the class finishes early)	15

1 Session Snapshot

Course	ISM3232 — Business Application Development
Session	Week 2 Lab — zsh Navigation & File Operations
Unit	Unit 1 · Developer Foundations
Class length	Full class period (75 minutes)
Format	Live terminal code-along
Prerequisites	Week 1: verified zsh/Python/Git, the ism3232/ folder structure (marked Midterm-Eligible)
Student-facing lab page	Week 2 In-Class Lab — Module 2A & 2B, “zsh Navigation and File Operations”
Parts covered	Part 1 (navigation) – Part 4 (command reference README) + Stretch (pipes/redirect)
Submission	4 screenshots + module02_zsh/README.md (12+ commands documented), Canvas, completion credit

This lab’s real subject is **muscle memory**, not new concepts — every command here is individually simple, and the actual teaching challenge is getting students to type each one deliberately, read its output before moving to the next, and build the reflex of checking *where they are* before doing anything destructive. Part 3’s `rm` safety ritual deserves outsized attention: it’s a two-command habit (`pwd` then `ls`, always, before `rm`) that this course will hold students to for the rest of the semester, and the earlier it becomes automatic, the fewer real losses happen later when `rm` targets something that actually matters.

2 Learning Objectives

By the end of this class period, students should be able to:

1. Navigate the filesystem using `pwd`, `ls`, `ls -la`, and `cd` (including `cd ..` and `cd ~`) without hesitation.
2. Visualize a folder’s structure with `tree -L 2`.
3. Create, copy, rename, read, and preview files with `touch`, `cp`, `mv`, `cat`, and `head`.
4. Apply the `rm` safety ritual — `pwd` then `ls`, every time, before any deletion — as an automatic habit, not a reminder they need every time.
5. Open a folder as a proper VS Code workspace, and document a session’s commands in a README table.

3 Before Class — Setup Checklist

- ☐ Confirm your own `ism3232/module02_zsh/` exists from Week 1 (or create it live if your section didn't reach that far) before demonstrating.
- ☐ Decide, and state explicitly at the start of Part 3, exactly when you will demonstrate a *real* `rm` — this is the one command all lab long with no undo, and it's worth being deliberate about narrating the safety ritual every single time you run it yourself today, modeling the exact habit being taught.
- ☐ Pre-check whether `tree` is installed on a representative sample of student machines (it's a common gap even on an otherwise-correct setup) — if missing, know the one-line fix for your platform (`brew install tree` on Mac) ready to share.

4 Materials Needed

- Terminal (zsh), VS Code, from Week 1's verified setup
- Students: their existing `ism3232/module02_zsh/` folder

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:04	Welcome + setup: create <code>week2_lab/</code>	4
0:04–0:20	Part 1 — Navigation	16
0:20–0:38	Part 2 — File operations	18
0:38–0:53	Part 3 — Safe file deletion	15
0:53–1:08	Part 4 — Command reference README	15
1:08–1:15	Stretch preview + wrap-up, submission checklist	7

Four required parts fill the class period; the pipes/redirect Stretch is positioned as a genuine bonus, since it introduces two new ideas (`|` and `>>`) beyond what the required parts cover, worth full attention only if time allows rather than rushed.

6 Segment-by-Segment Walkthrough

6.1 Setup (0:00–0:04)

Say to the class:

“Everything today happens inside a fresh week2_lab/ folder, so today’s practice doesn’t clutter what you’ll build in later weeks.”

Do, live:

```
cd ~/ism3232/module02_zsh
mkdir week2_lab
cd week2_lab
pwd
```

6.2 Part 1 — Navigation (0:04–0:20, 16 min)

Teaching goal: The four navigation commands (pwd, ls, ls -la, cd) fluent enough to use without thinking — and a first look at tree for visualizing structure at a glance.

Say to the class:

“Nine commands, in sequence. I want you reading each result before typing the next line — don’t paste this whole block at once. The point today is noticing what changes after each command.”

Live-code this, one command at a time, pausing for output after each:

```
pwd                # where am I right now?
ls                 # what visible files are here?
ls -la             # all files including hidden ones
cd ..              # up one level
pwd                # confirm new location
cd ~/ism3232       # jump to course folder
tree -L 2          # visualise the structure
cd module02_zsh/week2_lab # return to lab folder
pwd                # confirm
```

Line-by-line explanation:

- pwd / ls — Week 1’s familiar pair, now the automatic first move in any new terminal session — say explicitly: **making pwd your reflexive first command in any unfamiliar terminal state is exactly the habit this whole unit is building.**
- ls -la — two flags at once: -l (long format — one entry per line, with details like permissions

and size) and `-a` (all — including **hidden files**, whose names start with a dot, like `.gitignore` or `.zshrc`, which plain `ls` deliberately hides by default). Say explicitly why this matters: hidden files are not rare edge cases in this course — `.venv/`, `.gitignore`, and `.zshrc` are all coming in Week 3, and `ls -la` is how you'll confirm they exist.

- `cd ..` — `..` means “the parent of the current directory,” exactly like Module 01’s relative-path vocabulary in comparable courses; moves up exactly one level.
- `cd ~/ism3232` — an **absolute-from-home** jump, using `~` — contrast this explicitly with `cd ..`: `..` is always relative to wherever you currently are, while `~/ism3232` works identically no matter your starting location, since it’s anchored to the home directory.
- `tree -L 2` — **new command today**. `tree` draws the folder structure visually, as a text diagram; `-L 2` limits the depth shown to 2 levels, keeping the output readable rather than dumping every file in every nested subfolder. Say explicitly: this is the fastest way to get an at-a-glance sense of a project’s shape, and it becomes genuinely essential once projects have many nested folders (which this course’s own `ism3232/` structure already does).
- `cd module02_zsh/week2_lab` — a **relative path with two segments**, moving through two levels of nesting in one `cd` call — worth pointing out explicitly that `cd` isn’t limited to one folder at a time; a full relative (or absolute) path works exactly the same as a single folder name.

Verified example output (`tree -L 2` from `~/ism3232`, illustrative — exact contents will vary by what each student has built so far):

```
ism3232/
├── data
├── module01_setup
│   ├── hello_ism3232.py
│   └── README.md
├── module02_zsh
│   └── week2_lab
├── module03_git_github
├── module04_programming
├── module05_functions
├── module06_oop
├── module07_final_project
└── screenshots
```

Common student mistakes to watch for:

- Running `cd module02_zsh/week2_lab` from somewhere other than directly inside `~/ism3232` — produces `No such file or directory`, since this is a *relative* path requiring the right starting point; have the student run `pwd` first and reason about it, per the established habit.
- Confusing `tree`’s depth limit (`-L 2`) with somehow limiting *which* folders show, rather than *how deep* it looks — a good clarifying question: “if I ran `tree -L 1` instead, what would change?” (Only the top-level folder names would show, none of their contents — get a student to state this explicitly.)

- Not noticing hidden files in `ls -la`'s output because there simply aren't any yet at this point in the semester — reassure that this is expected; Week 3 is where `ls -la` starts showing something genuinely new (`.venv/`, `.gitignore`).

Check for understanding: “Starting from `~/ism3232/module02_zsh/week2_lab`, what's the shortest `cd` command to reach `~/ism3232/module01_setup`?” (`cd ../../module01_setup` — up two levels to `ism3232/`, then down into `module01_setup` — a good relative-path reasoning check, or accept `cd ~/ism3232/module01_setup` as an equally correct absolute alternative, and ask the student to name the tradeoff between the two styles.)

6.3 Part 2 — File Operations (0:20–0:38, 18 min)

Teaching goal: Create, write to, read, copy, and rename files entirely from the terminal, and open a folder properly as a VS Code workspace — the complete “manage files without a file browser” toolkit.

Say to the class:

“Six file operations, and I want you to genuinely read each output rather than just trusting the command ran. cat and head in particular — I want you to predict what each will print before you run it.”

Live-code this:

```
touch notes.txt commands.txt hello_week2.py
ls -la                                # confirm three files exist

echo 'Week 2 navigation practice' > notes.txt
cat notes.txt                         # print contents
head -1 notes.txt                     # first line

cp notes.txt notes_backup.txt         # copy
mv hello_week2.py week2_script.py    # rename
ls -la                                # confirm changes
```

Line-by-line explanation:

- `touch notes.txt commands.txt hello_week2.py` — Week 1’s touch, now creating **three** files in one command — same “multiple arguments” pattern as Part 2’s `mkdir` last week.
- `echo 'Week 2 navigation practice' > notes.txt` — **new syntax today: the > redirect operator**. `echo` alone would print its text to the terminal; `>` instead sends that output *into a file*, overwriting whatever was there before (an empty file, in this case). Say explicitly, since this is worth getting right early: **> completely replaces a file’s contents — this is the same “silent overwrite” risk as Module 12’s “w” file mode in a Python-focused course**, worth flagging now since the Stretch section’s `>>` is about to show the safer alternative.
- `cat notes.txt` — prints a file’s **entire contents** to the terminal — the standard, simplest way to quickly view a small file without opening an editor.
- `head -1 notes.txt` — prints only the **first** line (`-1` specifies “just one line”) — since `notes.txt` only has one line total, this looks identical to `cat`’s output right now; worth stating explicitly that the *real* value of `head` shows up on a longer file, where `cat`-ing the whole thing would flood the terminal.
- `cp notes.txt notes_backup.txt` — **copy**: `notes.txt` remains untouched, and a new file, `notes_backup.txt`, is created with identical contents. Contrast this explicitly with the next line.
- `mv hello_week2.py week2_script.py` — **rename** (or move, if given a different destination

folder rather than just a new name in the same location) — say explicitly: unlike `cp`, `mv` does **not** leave the original behind; after this line, `hello_week2.py` no longer exists at all, only `week2_script.py` does, with the same content.

- Then, in VS Code: add `print('Week 2 complete')` to `week2_script.py`, save, and run `python3 week2_script.py`.

Verified output (final `ls -la` should show four files: `commands.txt`, `notes.txt`, `notes_backup.txt`, `week2_script.py` — note `hello_week2.py` is genuinely gone, replaced by its renamed version):

```
Week 2 navigation practice
```

```
Week 2 navigation practice
```

```
Week 2 complete
```

(First two lines from `cat/head` respectively; third line from running the script after adding the `print()`.)

Common student mistakes to watch for:

- Confusing `cp`'s and `mv`'s argument order — both take source destination, i.e., “copy/move *this* to become *that*” — a reversed order copies/renames in the wrong direction; if this happens, have the student re-run `ls -la` and reason about which file now has which name/content before “fixing” it by guessing.
- Using `>` a second time on `notes.txt` by accident (e.g., re-running the `echo` line) — silently replaces the file's content again rather than adding to it; a good moment to preview that this is exactly the risk the Stretch section's `>>` avoids.
- Opening `week2_script.py` by double-clicking it in a file browser instead of via `code week2_script.py` or the already-open workspace — not an error, but a habit worth gently correcting, since it breaks the “everything through the terminal/workspace” discipline this unit is building.

Check for understanding: “If you ran `cp notes.txt notes.txt` — copying a file onto itself, same source and destination name — what would happen?” (Most systems either refuse with an error, like `cp: notes.txt and notes.txt are identical`, or leave the file unchanged — not something to demonstrate destructively, but a good reasoning question about what `cp`'s two-argument contract actually requires: a genuinely different destination.)

Then, open the folder properly in VS Code:

```
cd ~/ism3232/module02_zsh
code .
```

Confirm the Explorer panel shows `week2_lab/` and its files — reinforcing Week 1's “open the whole workspace, not individual files” habit, now one level up from last week.

6.4 Part 3 — Safe File Deletion (0:38–0:53, 15 min)

Teaching goal: `rm`'s permanence, and the two-step safety ritual (`pwd` then `ls`, every single time, before deleting) as an automatic reflex — this is the highest-stakes command taught all semester, and the only one with genuinely no undo.

Say to the class:

“Everything else today, if you make a mistake, you can fix it — rename it back, recreate it, copy it again. `rm` is different. There is no Trash, no Recycle Bin, no undo. Once it runs, the file is gone. So before every single `rm` for the rest of this course, you run two commands first: `pwd`, then `ls`. Confirm where you are, confirm what’s actually there. I’m going to do this myself, every time, all semester — watch me model it now.”

Live-code the full ritual, narrating each step’s purpose as you go:

```
# Step 1: confirm your location
pwd
```

```
# Step 2: confirm what is here
ls
```

```
# Now it is safe to delete
rm notes_backup.txt
```

```
# Step 3: confirm it is gone
ls
```

Line-by-line explanation:

- `pwd` — confirms you’re in the folder you *think* you’re in — say explicitly: this catches the scenario where an earlier `cd` didn’t do what you expected, and you’re about to delete something in the wrong place entirely.
- `ls` — confirms the file you’re about to delete actually exists here, spelled the way you expect, and — just as importantly — lets you see *what else* is here, so a typo in the `rm` command’s filename doesn’t silently delete the wrong thing.
- `rm notes_backup.txt` — the actual deletion. No confirmation prompt, no warning, no Trash — say plainly, one more time, since repetition is the point: **this is permanent**.
- Final `ls` — confirms the deletion happened, and that *only* the intended file is gone, nothing else.

Verified output (the file count in the final `ls` should be exactly one fewer than before):

```
/Users/yourname/ism3232/module02_zsh/week2_lab
commands.txt  notes.txt    notes_backup.txt  week2_script.py
commands.txt  notes.txt    week2_script.py
```

Common student mistakes to watch for:

- Skipping straight to `rm` without the `pwd/ls` steps, especially once students feel confident and start moving faster — this is worth actively watching for and correcting every time you see it during this specific part, even though it slows the room down; the entire point is that the ritual becomes reflexive *before* speed does.
- Using a wildcard carelessly (e.g., `rm *` or `rm *.txt`) without fully understanding what it will match — not required by this exercise, but worth a strong, explicit warning if any student is already experimenting with wildcards on their own: `rm *.txt` deletes **every** `.txt` file in the current directory at once, with the same total permanence as a single-file `rm`. If you demonstrate this concept at all, do it as a verbal warning, not a live demo on a real folder.
- Attempting to `rm` a folder with the same syntax as a file — `rm` alone refuses on a directory (`rm: notes_backup.txt: is a directory` — no, actually the correct message is `rm: some_folder: is a directory` when attempted); removing a folder requires `rm -r` (recursive), which is **more** dangerous, not less, since it deletes everything inside, and is worth mentioning only as a “there is a version of this command that’s even more permanent, and this course does not use it lightly” caution rather than demonstrating live.

Check for understanding: “Why does the ritual specifically require *both* `pwd` and `ls`, rather than just one or the other?” (`pwd` confirms *where* you are; `ls` confirms *what’s actually there* — a student could be in the exact right folder but still misremember or mistype a filename, and `ls` catches that; conversely, a student could correctly see a familiar-looking filename in `ls` output while actually being in the wrong folder entirely, which `pwd` alone would have caught. Neither check alone covers both failure modes — this is worth having a student articulate in their own words, not just recite.)

6.5 Part 4 — Command Reference README (0:53–1:08, 15 min)

Teaching goal: Document every command practiced today in a structured README table — reinforcing both the commands themselves (writing a description forces genuine recall, not just muscle memory) and the documentation habit established in Week 1.

Say to the class:

“Twelve commands, minimum, each with a one-line description in your own words. Writing the description is the actual test of whether you understand what a command does, not just that you can type it.”

Do, live:

```
cd ~/ism3232/module02_zsh
touch README.md
code README.md
```

Write this content, filling in the full command list:

```
# ISM3232 - Module 2: zsh Navigation and File Operations
```

```
## Commands Practiced
```

Command	What it does
pwd	Prints the current working directory
ls	Lists visible files and folders
ls -la	Lists all files including hidden ones
[add the rest]	[your description]

```
## AI Use Statement
```

```
I did not use AI for this lab.
```

Line-by-line explanation:

- The table format is identical Markdown syntax to Week 1’s version table — pipes (|) separate columns, a row of dashes (|---|---|) marks the header boundary — worth reinforcing rather than re-teaching, since this is the second week in a row using the same table syntax.
- **“Document at least 12 commands from the lab”** — say explicitly: this isn’t an arbitrary number — between Parts 1 through 3, students have genuinely used at least that many distinct commands (pwd, ls, ls -la, cd, tree, touch, echo, cat, head, cp, mv, rm, plus code and python3 if counted) — the requirement is really asking “did you document *everything* you did today,” not “pad the list to reach a number.”
- Each description should be written **from memory first, then verified** — say explicitly: the value of this exercise comes from attempting a description before checking, since that’s what

actually tests recall; looking up every single one immediately defeats the purpose.

Common student mistakes to watch for:

- Descriptions that just restate the command name without explaining its function (“`ls -la` — lists files with `-la`”) — push for genuine explanation: what does `-la` specifically add over plain `ls`, in the student’s own words.
- Forgetting to include `code` and `python3` in the count, since they might not feel like “navigation” commands the way `cd/ls` do — remind the room that any command genuinely used today counts, not just the ones from Parts 1–3’s headline lists.
- Copying descriptions verbatim from this guide or the lab page rather than writing original phrasing — a brief, low-key reminder that the AI Use Statement below the table is specifically about honesty regarding how the work was actually produced.

Check for understanding: “Which command from today would you have the hardest time explaining to someone who’s never used a terminal, and why?” (No single right answer — a good closing discussion question that gets students to genuinely reflect on which concepts (redirect operators, hidden files, the distinction between `cp` and `mv`) felt least automatic yet, which is useful both for the student’s own awareness and for you as a signal of what to reinforce next week.)

6.6 Stretch — Pipes and Redirect (1:08–1:15, as time allows)

Frame as a genuine bonus, worth full attention only if time allows:

```
ls -la | head -5           # list, pipe into head
echo 'extra line' >> notes.txt # append (>> not >)
cat notes.txt              # confirm both lines
wc -l notes.txt            # count lines
```

If you demo this live, two ideas are worth stating explicitly:

- `ls -la | head -5` — the **pipe** (`|`) takes the *output* of the command on its left and feeds it as *input* to the command on its right — here, `ls -la`'s full listing gets fed into `head -5`, showing only the first 5 lines of that listing rather than everything. Say plainly: **piping is how you chain simple commands together into something more specific**, without needing a single command that does exactly what you want built in — this is a foundational idea in terminal/Unix workflows, genuinely worth a few extra minutes if the room has them.
- `echo 'extra line' >> notes.txt` — **two > characters, not one — this is the fix to Part 2's overwrite risk**. `>>` **appends** to a file, adding a new line at the end, while preserving everything already there; a single `>` (Part 2's version) would have erased `notes.txt`'s existing content entirely. Ask the room to predict `cat notes.txt`'s output *before* running it — a good check that the append-vs-overwrite distinction actually landed. `wc -l` (word count, `-l` for lines specifically) then confirms the file now has two lines, not one.

7 Wrap-Up (last ~7 minutes)

Review the submission checklist together, on screen:

- ☐ **Screenshot 1:** `tree -L 2` output from `~/ism3232/`
- ☐ **Screenshot 2:** `ls -la` and `python3 week2_script.py` output
- ☐ **Screenshot 3:** VS Code with `module02_zsh` open as workspace
- ☐ **Screenshot 4:** `pwd` + `ls` before `rm`, then `ls` after — all in one screenshot
- ☐ **File upload:** `module02_zsh/README.md` — command reference table with at least 12 commands
- ☐ All uploaded to Canvas by end of class

Preview Week 3: “Every hidden file you glimpsed in `ls -la` today with nothing actually there yet — that changes next week. Virtual environments and shell customization mean your `ls -la` output finally has something real to show.”

8 Appendix A — Full Command Reference

Setup:

```
cd ~/ism3232/module02_zsh
mkdir week2_lab
cd week2_lab
pwd
```

Part 1 (navigation):

```
pwd
ls
ls -la
cd ..
pwd
cd ~/ism3232
tree -L 2
cd module02_zsh/week2_lab
pwd
```

Part 2 (file operations):

```
touch notes.txt commands.txt hello_week2.py
ls -la
echo 'Week 2 navigation practice' > notes.txt
cat notes.txt
head -1 notes.txt
cp notes.txt notes_backup.txt
mv hello_week2.py week2_script.py
```

```
ls -la
```

```
# add to week2_script.py:  
print('Week 2 complete')
```

```
python3 week2_script.py  
cd ~/ism3232/module02_zsh  
code .
```

Part 3 (safe deletion):

```
pwd  
ls  
rm notes_backup.txt  
ls
```

Part 4 (module02_zsh/README.md): see the full template in the Part 4 walkthrough above; minimum 12 commands.

Stretch (pipes/redirect):

```
ls -la | head -5  
echo 'extra line' >> notes.txt  
cat notes.txt  
wc -l notes.txt
```

9 Appendix B — Extra Practice (only if the class finishes early)

Four required parts fill the full class period at a normal pace. If a section moves unusually fast:

Extra — a second navigation round, deeper nesting. Have students create `week2_lab/practice/deep/folder` (three nested `mkdirs` or `mkdir -p` in one step) and practice reaching it three different ways: a full relative path from `week2_lab/` in one `cd`, three separate single-level `cds`, and an absolute path from `~`. Good rehearsal of the relative-vs-absolute reasoning from Part 1's check-for-understanding.

Extra — a second safe-deletion round. Have students create a throwaway file, run the full `pwd/ls/rm/ls` ritual on it independently (not narrated by the instructor this time), and screenshot it themselves — a good, low-stakes independent rehearsal of the exact habit Part 3 is building, before it's needed for real later in the semester.